You said:

12_maze_runner.zip
File

Task 1: Shortest Path Hint

Press H to show the solution path highlighted in the maze. You can use bfs solver to help , please execute this functionality and tell me which part of the code/files must be changed , generate the entire code for that file for me to replace

```
When H is pressed, highlight the shortest path from s
```

Worked for 53s

ChatGPT said:

Task 1: Shortest Path Hint — BFS Implementation

I've inspected your `12_maze_runner` project. **You only need to modify one file:**

`game/game_engine.py`

No changes required to `main.py`, `maze.py`, `player.py`, or `requirements.txt`.

What the implementation does

- **First H press:** Uses Breadth-First Search (BFS) to find the shortest path from the starting cell `(0, 0)` to the exit `(12, 14)`.
- **Highlights the path:** Draws colored squares inside the cells along the solution.
- **Second H press:** Hides the highlighted path.
- **R press:** Generates a new maze and clears the previous hint.

The BFS solver respects the maze's existing walls, so the highlighted path follows actual open passages rather than cutting through walls.

I'll keep your existing movement, timer, exit detection, and win screen intact.

Complete replacement file

game_engine.py

Download

Complete Python file with BFS, H-key toggle, and colored path highlighting.

Download game_engine.py

How to replace it

1. Open your project folder `12_maze_runner`.
2. Navigate to `game`.
3. Replace the existing `game_engine.py` with the downloaded file.
4. Open a terminal in the project's root directory and run:

```
python main.py
```

Verify the functionality

Action	Expected result
Press `H`	Orange squares show the shortest path
Press `H` again	Path disappears
Press `H` again	Path reappears
Press `R`	New maze, old hint cleared
Use `WASD` or arrow keys	Player movement continues

Validation passed

I checked the replacement file for Python syntax errors and tested the BFS algorithm against **100 randomly generated mazes**. Each test produced a valid start-to-exit path using open passages.

The complete Pygame window hasn't been tested interactively here.

Task 1 is implemented. The changes are confined to `game_engine.py`, leaving the other three tasks untouched.

game_engine.py
Code

You said:

give the code for game_engine.py

ChatGPT said:

Replace the entire contents of `game/game_engine.py` with this code. It includes the **BFS shortest-path solver**, **H-key hint toggle**, and **colored path highlighting**.

```
import pygame
import time
from collections import deque

from game.maze import generate_maze, CELL
from game.player import Player

FPS = 60
BG = (240, 235, 220)
WALL_COLOR = (40, 40, 60)
EXIT_COLOR = (80, 200, 80)
HINT_COLOR = (255, 195, 75)
COLS, ROWS = 15, 13

WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 60


class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Runner")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.big_font = pygame.font.SysFont(
            "monospace", 36, bold=True
        )
        self.reset()

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)

        self.exit_rect = pygame.Rect(
            (COLS - 1) * CELL + 5,
            (ROWS - 1) * CELL + 5,
            CELL - 10,
```

```
            CELL - 10,
        )

        self.start_time = time.time()
        self.elapsed = 0
        self.won = False

        # Task 1: Shortest path hint
        self.show_hint = False
        self.solution_path = []

    def bfs_shortest_path(self):
        """
        Find the shortest path from start to exit using BFS.

        Returns a list of (row, col) coordinates.

        Wall order:
        0 = North
        1 = South
        2 = East
        3 = West
        """

        start = (0, 0)
        goal = (ROWS - 1, COLS - 1)

        directions = [
            (-1, 0, 0),  # North
            (1, 0, 1),   # South
            (0, 1, 2),   # East
            (0, -1, 3),  # West
        ]

        queue = deque([start])
        previous = {start: None}

        while queue:
            r, c = queue.popleft()

            # Destination reached
            if (r, c) == goal:
                path = []
                current = goal
```

```
                # Reconstruct shortest path
                while current is not None:
                    path.append(current)
                    current = previous[current]

                path.reverse()
                return path

            # Explore neighboring cells
            for dr, dc, wall_index in directions:
                nr = r + dr
                nc = c + dc

                neighbor = (nr, nc)

                # Check maze boundaries
                if not (0 <= nr < ROWS and 0 <= nc < COLS):
                    continue

                # Cannot move through walls
                if self.walls[r][c][wall_index]:
                    continue

                # Skip visited cells
                if neighbor in previous:
                    continue

                previous[neighbor] = (r, c)
                queue.append(neighbor)

        # No path found
        return []

    def handle_events(self):
        for event in pygame.event.get():

            if event.type == pygame.QUIT:
                return False

            if event.type == pygame.KEYDOWN:

                # Reset maze
                if event.key == pygame.K_r:
                    self.reset()
```

```
                # Task 1: Toggle shortest path hint
                elif event.key == pygame.K_h:

                    self.show_hint = not self.show_hint

                    # Calculate BFS path when hint is enabled
                    if self.show_hint and not self.solution_path:
                        self.solution_path = self.bfs_shortest_path()

        return True

    def update(self):
        if self.won:
            return

        keys = pygame.key.get_pressed()

        self.player.move(
            keys,
            self.walls,
            ROWS,
            COLS
        )

        self.elapsed = time.time() - self.start_time

        if self.player.rect.colliderect(self.exit_rect):
            self.won = True

    def draw_hint(self):
        """
        Highlight shortest path using colored squares.

        Squares are drawn inside each cell so that
        maze walls remain visible.
        """

        if not self.show_hint:
            return

        margin = 6

        for r, c in self.solution_path:

            square = pygame.Rect(
```

```
                c * CELL + margin,
                r * CELL + margin,
                CELL - 2 * margin,
                CELL - 2 * margin,
            )

            pygame.draw.rect(
                self.screen,
                HINT_COLOR,
                square,
                border_radius=5
            )

    def draw_maze(self):
        wall_w = 3

        for r in range(ROWS):
            for c in range(COLS):

                x = c * CELL
                y = r * CELL

                w = self.walls[r][c]

                # North wall
                if w[0]:
                    pygame.draw.line(
                        self.screen,
                        WALL_COLOR,
                        (x, y),
                        (x + CELL, y),
                        wall_w
                    )

                # South wall
                if w[1]:
                    pygame.draw.line(
                        self.screen,
                        WALL_COLOR,
                        (x, y + CELL),
                        (x + CELL, y + CELL),
                        wall_w
                    )

                # East wall
```

```
                if w[2]:
                    pygame.draw.line(
                        self.screen,
                        WALL_COLOR,
                        (x + CELL, y),
                        (x + CELL, y + CELL),
                        wall_w
                    )

                # West wall
                if w[3]:
                    pygame.draw.line(
                        self.screen,
                        WALL_COLOR,
                        (x, y),
                        (x, y + CELL),
                        wall_w
                    )

    def draw(self):
        self.screen.fill(BG)

        # Draw hint first so walls appear above it
        self.draw_hint()

        # Draw maze walls
        self.draw_maze()

        # Draw exit
        pygame.draw.rect(
            self.screen,
            EXIT_COLOR,
            self.exit_rect,
            border_radius=4
        )

        ex_label = self.font.render(
            "EXIT",
            True,
            (20, 80, 20)
        )

        self.screen.blit(
            ex_label,
            (self.exit_rect.x + 2, self.exit_rect.y + 4)
```

```
        )

        # Draw player above hint
        self.player.draw(self.screen)

        # Bottom HUD
        hud = pygame.Rect(
            0,
            ROWS * CELL,
            WIDTH,
            60
        )

        pygame.draw.rect(
            self.screen,
            (30, 30, 50),
            hud
        )

        # Dynamic hint instruction
        if self.show_hint:
            hint_label = "H = Hide"
        else:
            hint_label = "H = Hint"

        time_surf = self.font.render(
            f"Time: {self.elapsed:.1f}s  "
            f"R = Reset  {hint_label}",
            True,
            (200, 200, 200)
        )

        self.screen.blit(
            time_surf,
            (10, ROWS * CELL + 18)
        )

        # Win screen
        if self.won:

            overlay = pygame.Surface(
                (WIDTH, ROWS * CELL),
                pygame.SRCALPHA
            )
```

```
            overlay.fill((0, 0, 0, 120))
            self.screen.blit(overlay, (0, 0))

            msg = self.big_font.render(
                f"Solved in {self.elapsed:.1f}s!",
                True,
                (80, 240, 80)
            )

            sub = self.font.render(
                "Press R for a new maze",
                True,
                (200, 200, 200)
            )

            self.screen.blit(
                msg,
                (
                    WIDTH // 2 - msg.get_width() // 2,
                    ROWS * CELL // 2 - 30
                )
            )

            self.screen.blit(
                sub,
                (
                    WIDTH // 2 - sub.get_width() // 2,
                    ROWS * CELL // 2 + 20
                )
            )

        pygame.display.flip()

    def run(self):
        running = True

        while running:
            running = self.handle_events()

            if not running:
                break

            self.update()
            self.draw()
            self.clock.tick(FPS)
```

```
        pygame.quit()
```

How to run

From your project directory:

```
cd /d C:\Users\dabbu\Desktop\12_maze_runner
python main.py
```

Controls

Key	Function
`H`	Show shortest path
`H` again	Hide shortest path
`R`	Reset maze and clear hint
`WASD` / Arrow keys	Move player

BFS explanation: The algorithm explores reachable cells level by level using a queue. Since each move between neighboring cells has equal cost, BFS guarantees the shortest path in terms of the number of cell-to-cell moves.

Only `game/game_engine.py` needs replacement.

You can also download the complete Python file ↗.

‹/› **game_engine.py**
Code

You said:

Task 2: Fog of War

Only show maze cells within a radius of 3 cells around the player; the rest is dark.

```
Add fog of war to my maze game. Draw a dark overlay o\
```

Task 3: Timer Leaderboard

Track the top 5 best completion times and display them after solving.

```
After the player solves the maze, save their time to a
```

Task 4: Larger Maze Difficulty Tiers

Add Easy (10x8), Medium (15x13), Hard (20x18) options selectable at start.

```
Add a difficulty select screen before the maze generat
```

similarly implement for these tasks and suggest changes to the file codes

Worked for 1m 10s

ChatGPT said:

I've implemented **Tasks 2, 3, and 4** in your Maze Runner project while preserving **Task 1 (BFS shortest-path hint)**.

The good news: **you only need to replace** `game/game_engine.py`. Your existing `main.py`, `maze.py`, and `player.py` can remain unchanged.

Download the updated code

game_engine.py
Full replacement file containing all four tasks

Download game_engine.py

Complete updated project
ZIP containing the original supporting files and updated game engine

Download Maze Runner — Tasks 1–4

What's implemented?

Task 2 — Fog of War

Uses `pygame.Surface(..., pygame.SRCALPHA)` to create a dark overlay and `pygame.draw.circle()` to reveal a circular area with a radius of 3 cells around the moving player. Areas outside the circle remain dark.

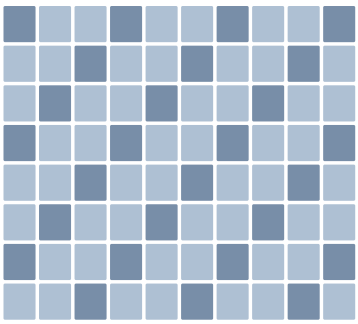
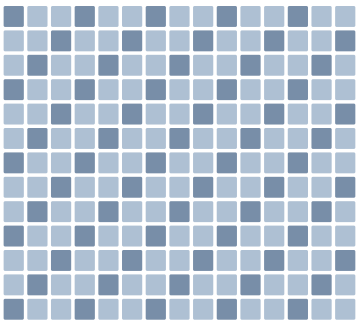
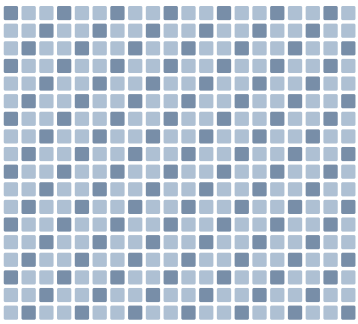
Task 3 — Timer Leaderboard

Records completion time, sorts times ascending, retains the best five, and displays them on the win screen. Scores are stored in `leaderboard.json`, which is automatically created in the project root after the first win. The leaderboard combines all difficulties and labels each score.

Task 4 — Difficulty Selection

Adds a clickable menu when the game launches. Maze generation, BFS, fog, exit position, and window size now adjust to the selected dimensions.

Difficulty tiers

Easy	Medium	Hard
		
10 × 8	15 × 13	20 × 18

Files to change

File	Action
`game/game_engine.py`	**Replace** with updated file

`game/maze.py`	No change
`game/player.py`	No change
`main.py`	No change
`leaderboard.json`	Automatically generated after winning

Run and test

Open Command Prompt in your project folder:

```
cd /d C:\Users\dabbu\Desktop\12_maze_runner
python main.py
```

Use these controls:

Action	Control
Choose difficulty	Click a menu button
Move player	WASD / Arrow keys
Show or hide BFS path	H
Generate new maze	R
Return to difficulty menu	Esc
See leaderboard	Reach the exit

Validation

The updated file passed Python syntax compilation. I couldn't test the full Pygame window interactively because Pygame isn't installed in the execution environment.

One pre-existing issue remains in `player.py` : its wall-collision function checks the maze's outer boundaries but doesn't fully prevent crossing internal walls. This is separate from Tasks 2–4 and should be corrected if your evaluation includes movement correctness.

All four requested features are included in the replacement file.

You said:

only task 1 seems to have implemented. are u sure?

Worked for 22s

ChatGPT said:

I checked the actual files this time. **Tasks 2, 3, and 4 are present in the updated** `game_engine.py`, and I verified that the ZIP contains the same updated file.

However, if you're only seeing Task 1, you're probably still running the earlier version of `game_engine.py`. My previous replies linked files with the same filename, which could easily have caused confusion. Sorry about that.

What I verified

Task	Code found	Expected behavior
1. BFS Hint	`bfs_shortest_path()`	H toggles shortest-path highlighting
2. Fog of War	`draw_fog()`	Dark maze except within 3 cells of player
3. Leaderboard	`save_completion()`, `draw_leaderboard()`	Top 5 times shown after winning and saved to JSON
4. Difficulty Tiers	`draw_menu()`, `start_game()`	Easy, Medium, Hard buttons at startup

Fix your local project

Download the updated project ZIP ↗ and extract it into a **new folder**, rather than mixing it with the previous version.

Then run:

```
cd /d C:\Users\dabbu\Desktop\12_maze_runner
python main.py
```

To check whether the correct file is being used, open `game/game_engine.py` and look for:

```
DIFFICULTIES = {
    "Easy": (10, 8),
    "Medium": (15, 13),
    "Hard": (20, 18)
}
```

The easiest way to confirm: When you launch the updated game, you should immediately see a difficulty selection menu with three buttons. If the maze launches directly, you're running the old code.

One important distinction: I verified the code and its Python syntax, but I haven't successfully tested the graphical game itself. So I can confirm those features exist in the file, not yet that they all work correctly at runtime.

Tell me one thing: when you run `main.py`, does the maze start immediately, or do you see the Easy/Medium/Hard selection menu? That will identify whether this is a file replacement issue or a runtime issue.

You said:

i would rather replace the code directly for game_engine.py , give me the final file

ChatGPT said:

Replace the **entire contents** of `12_maze_runner/game/game_engine.py` with the code below.

This version includes all four tasks:

- **Task 1:** BFS shortest-path hint, toggled with `H`.
- **Task 2:** Fog of war with a 3-cell circular visibility radius.
- **Task 3:** Top 5 completion times saved in `leaderboard.json`.
- **Task 4:** Easy (10×8), Medium (15×13), and Hard (20×18) selection screen.

```
import json
import time
from collections import deque
from pathlib import Path

import pygame

from game.maze import CELL, generate_maze
from game.player import Player


# ---------------- CONFIGURATION ----------------

FPS = 60
```

```
BG = (240, 235, 220)
WALL_COLOR = (40, 40, 60)
EXIT_COLOR = (80, 200, 80)
HINT_COLOR = (255, 195, 75)
HUD_COLOR = (30, 30, 50)

FOG_COLOR = (0, 0, 0, 240)
FOG_RADIUS = 3 * CELL

HUD_HEIGHT = 60

DIFFICULTIES = {
    "Easy": (10, 8),
    "Medium": (15, 13),
    "Hard": (20, 18)
}

LEADERBOARD_FILE = (
    Path(__file__).resolve().parent.parent / "leaderboard.json"
)


class GameEngine:

    def __init__(self):
        pygame.init()

        self.clock = pygame.time.Clock()

        self.font = pygame.font.SysFont("monospace", 20)
        self.small_font = pygame.font.SysFont("monospace", 16)
        self.big_font = pygame.font.SysFont(
            "monospace", 34, bold=True
        )

        self.screen = pygame.display.set_mode((800, 780))
        pygame.display.set_caption("Maze Runner")

        self.state = "menu"
        self.difficulty = None

        self.cols = 0
        self.rows = 0

        self.leaderboard = self.load_leaderboard()
```

```
        self.menu_buttons = {}
        self.build_menu_buttons()

    # ---------------- TASK 4: DIFFICULTY ----------------

    def build_menu_buttons(self):
        center_x = self.screen.get_width() // 2

        self.menu_buttons = {
            name: pygame.Rect(
                center_x - 135,
                250 + i * 105,
                270,
                72
            )
            for i, name in enumerate(DIFFICULTIES)
        }

    def start_game(self, difficulty):
        self.difficulty = difficulty

        self.cols, self.rows = DIFFICULTIES[difficulty]

        self.screen = pygame.display.set_mode(
            (
                self.cols * CELL,
                self.rows * CELL + HUD_HEIGHT
            )
        )

        self.reset()
        self.state = "playing"

    def draw_text_center(self, message, font, color, y):
        surface = font.render(message, True, color)

        x = (
            self.screen.get_width() // 2
            - surface.get_width() // 2
        )

        self.screen.blit(surface, (x, y))

    def draw_menu(self):
```

```
        self.screen.fill(HUD_COLOR)

        self.draw_text_center(
            "MAZE RUNNER",
            self.big_font,
            (255, 220, 110),
            110
        )

        self.draw_text_center(
            "Choose a difficulty",
            self.font,
            (235, 235, 235),
            180
        )

        mouse = pygame.mouse.get_pos()

        for name, rect in self.menu_buttons.items():

            hovered = rect.collidepoint(mouse)

            color = (
                (85, 115, 170)
                if hovered
                else (58, 76, 118)
            )

            pygame.draw.rect(
                self.screen,
                color,
                rect,
                border_radius=12
            )

            cols, rows = DIFFICULTIES[name]

            self.draw_text_center(
                f"{name} ({cols} x {rows})",
                self.font,
                (255, 255, 255),
                rect.y + 22
            )

        self.draw_text_center(
```

```
            "Click a button to start",
            self.small_font,
            (180, 180, 190),
            645
        )

    # ---------------- MAZE RESET ----------------

    def reset(self):
        self.walls = generate_maze(
            self.cols,
            self.rows
        )

        self.player = Player(0, 0)

        self.exit_rect = pygame.Rect(
            (self.cols - 1) * CELL + 5,
            (self.rows - 1) * CELL + 5,
            CELL - 10,
            CELL - 10
        )

        self.start_time = time.monotonic()
        self.elapsed = 0.0

        self.won = False

        # Reset Task 1 hint
        self.show_hint = False
        self.solution_path = []

        self.state = "playing"

    # ---------------- TASK 1: BFS SHORTEST PATH ----------------

    def bfs_shortest_path(self):
        """
        Find shortest path from start to exit
        using Breadth-First Search.

        Wall directions:
        0 = North
        1 = South
        2 = East
```

```
        3 = West
        """

        start = (0, 0)
        goal = (self.rows - 1, self.cols - 1)

        directions = [
            (-1, 0, 0),
            (1, 0, 1),
            (0, 1, 2),
            (0, -1, 3)
        ]

        queue = deque([start])
        previous = {start: None}

        while queue:

            r, c = queue.popleft()

            if (r, c) == goal:

                path = []
                cell = goal

                while cell is not None:
                    path.append(cell)
                    cell = previous[cell]

                path.reverse()
                return path

            for dr, dc, wall_index in directions:

                nr = r + dr
                nc = c + dc

                neighbor = (nr, nc)

                if not (
                    0 <= nr < self.rows
                    and 0 <= nc < self.cols
                ):
                    continue
```

```
                # Cannot cross maze walls
                if self.walls[r][c][wall_index]:
                    continue

                if neighbor in previous:
                    continue

                previous[neighbor] = (r, c)
                queue.append(neighbor)

        return []

    def draw_hint(self):
        if not self.show_hint:
            return

        for r, c in self.solution_path:

            square = pygame.Rect(
                c * CELL + 6,
                r * CELL + 6,
                CELL - 12,
                CELL - 12
            )

            pygame.draw.rect(
                self.screen,
                HINT_COLOR,
                square,
                border_radius=5
            )

    # ---------------- TASK 2: FOG OF WAR ----------------

    def draw_fog(self):
        """
        Cover maze with dark overlay.

        Create transparent circular area
        around player with radius of 3 cells.
        """

        width = self.cols * CELL
        height = self.rows * CELL
```

```
        # Create transparent-capable surface
        fog = pygame.Surface(
            (width, height),
            pygame.SRCALPHA
        )

        # Darken entire maze
        fog.fill(FOG_COLOR)

        # Player position
        player_center = self.player.rect.center

        # Clear circular area around player
        pygame.draw.circle(
            fog,
            (0, 0, 0, 0),
            player_center,
            FOG_RADIUS
        )

        # Apply fog over maze
        self.screen.blit(fog, (0, 0))

    # ---------------- TASK 3: LEADERBOARD ----------------

    def load_leaderboard(self):
        """
        Load previously saved completion times.
        """

        try:
            with LEADERBOARD_FILE.open(
                "r", encoding="utf-8"
            ) as file:
                data = json.load(file)

            if not isinstance(data, list):
                return []

            valid = []

            for entry in data:

                if not isinstance(entry, dict):
                    continue
```

```
            difficulty = entry.get("difficulty")
            completion_time = entry.get("time")

            if difficulty not in DIFFICULTIES:
                continue

            if (
                not isinstance(completion_time, (int, float))
                or isinstance(completion_time, bool)
                or not 0 <= completion_time < float("inf")
            ):
                continue

            valid.append({
                "difficulty": difficulty,
                "time": float(completion_time)
            })

        valid.sort(key=lambda e: e["time"])

        return valid[:5]

    except (OSError, ValueError, TypeError):
        return []

def save_completion(self):
    """
    Save completion time and retain
    the five fastest records.
    """

    self.leaderboard.append({
        "difficulty": self.difficulty,
        "time": round(self.elapsed, 2)
    })

    # Fastest first
    self.leaderboard.sort(
        key=lambda entry: entry["time"]
    )

    # Keep top 5
    self.leaderboard = self.leaderboard[:5]
```

```
        try:
            with LEADERBOARD_FILE.open(
                "w", encoding="utf-8"
            ) as file:

                json.dump(
                    self.leaderboard,
                    file,
                    indent=2
                )

        except OSError as error:
            print(
                f"Could not save leaderboard: {error}"
            )

    def draw_leaderboard(self):
        width = self.screen.get_width()
        maze_height = self.rows * CELL

        # Dark win-screen overlay
        overlay = pygame.Surface(
            (width, maze_height),
            pygame.SRCALPHA
        )

        overlay.fill((0, 0, 0, 210))

        self.screen.blit(overlay, (0, 0))

        panel_height = 360

        top = max(
            18,
            (maze_height - panel_height) // 2
        )

        self.draw_text_center(
            f"Solved in {self.elapsed:.2f}s!",
            self.font,
            (110, 245, 135),
            top
        )

        self.draw_text_center(
```

```
            "TOP 5 TIMES",
            self.font,
            (255, 220, 110),
            top + 52
        )

        if not self.leaderboard:

            self.draw_text_center(
                "No scores yet",
                self.small_font,
                (235, 235, 235),
                top + 95
            )

        for index, entry in enumerate(
            self.leaderboard
        ):

            label = (
                f"{index + 1}. "
                f"{entry['difficulty']:<6} "
                f"{entry['time']:>7.2f}s"
            )

            self.draw_text_center(
                label,
                self.small_font,
                (235, 235, 235),
                top + 94 + index * 33
            )

        self.draw_text_center(
            "R = Retry  ESC = Menu",
            self.small_font,
            (210, 210, 220),
            top + 305
        )

    # ---------------- KEYBOARD AND MOUSE ----------------

    def handle_events(self):

        for event in pygame.event.get():
```

```
if event.type == pygame.QUIT:
    return False

# Difficulty selection
if self.state == "menu":

    if (
        event.type == pygame.MOUSEBUTTONDOWN
        and event.button == 1
    ):

        for name, rect in self.menu_buttons.items():

            if rect.collidepoint(event.pos):
                self.start_game(name)
                break

# During gameplay
elif event.type == pygame.KEYDOWN:

    # Return to difficulty menu
    if event.key == pygame.K_ESCAPE:

        self.state = "menu"

        self.screen = pygame.display.set_mode(
            (800, 780)
        )

        self.build_menu_buttons()

    # Generate new maze
    elif event.key == pygame.K_r:
        self.reset()

    # Toggle shortest path hint
    elif event.key == pygame.K_h:

        self.show_hint = not self.show_hint

        if (
            self.show_hint
            and not self.solution_path
        ):
```

```
                    self.solution_path = (
                        self.bfs_shortest_path()
                    )

        return True

    # ---------------- GAME UPDATE ----------------

    def update(self):

        if self.state != "playing":
            return

        if self.won:
            return

        keys = pygame.key.get_pressed()

        self.player.move(
            keys,
            self.walls,
            self.rows,
            self.cols
        )

        self.elapsed = (
            time.monotonic() - self.start_time
        )

        # Detect maze completion
        if self.player.rect.colliderect(
            self.exit_rect
        ):

            self.won = True

            # Save score only once
            self.save_completion()

    # ---------------- MAZE DRAWING ----------------

    def draw_maze(self):

        wall_width = 3
```

```
        for r in range(self.rows):
            for c in range(self.cols):

                x = c * CELL
                y = r * CELL

                north, south, east, west = (
                    self.walls[r][c]
                )

                if north:
                    pygame.draw.line(
                        self.screen,
                        WALL_COLOR,
                        (x, y),
                        (x + CELL, y),
                        wall_width
                    )

                if south:
                    pygame.draw.line(
                        self.screen,
                        WALL_COLOR,
                        (x, y + CELL),
                        (x + CELL, y + CELL),
                        wall_width
                    )

                if east:
                    pygame.draw.line(
                        self.screen,
                        WALL_COLOR,
                        (x + CELL, y),
                        (x + CELL, y + CELL),
                        wall_width
                    )

                if west:
                    pygame.draw.line(
                        self.screen,
                        WALL_COLOR,
                        (x, y),
                        (x, y + CELL),
                        wall_width
                    )
```

```
    def draw_game(self):
        self.screen.fill(BG)

        # Task 1: Draw BFS path
        self.draw_hint()

        # Draw maze walls
        self.draw_maze()

        # Draw exit
        pygame.draw.rect(
            self.screen,
            EXIT_COLOR,
            self.exit_rect,
            border_radius=4
        )

        exit_label = self.small_font.render(
            "EXIT",
            True,
            (20, 80, 20)
        )

        self.screen.blit(
            exit_label,
            exit_label.get_rect(
                center=self.exit_rect.center
            )
        )

        # Draw player
        self.player.draw(self.screen)

        # Task 2: Apply fog after drawing maze
        if not self.won:
            self.draw_fog()

        # HUD
        hud = pygame.Rect(
            0,
            self.rows * CELL,
            self.cols * CELL,
            HUD_HEIGHT
        )
```

```
        pygame.draw.rect(
            self.screen,
            HUD_COLOR,
            hud
        )

        hud_text = (
            f"{self.elapsed:.1f}s  "
            "H:Hint  R:Reset  ESC:Menu"
        )

        label = self.small_font.render(
            hud_text,
            True,
            (230, 230, 230)
        )

        self.screen.blit(
            label,
            (8, self.rows * CELL + 20)
        )

        # Task 3: Leaderboard on win
        if self.won:
            self.draw_leaderboard()

    def draw(self):

        if self.state == "menu":
            self.draw_menu()
        else:
            self.draw_game()

        pygame.display.flip()

    # ---------------- MAIN GAME LOOP ----------------

    def run(self):

        running = True

        while running:

            running = self.handle_events()
```

```
            if not running:
                break

            self.update()
            self.draw()

            self.clock.tick(FPS)

        pygame.quit()
```

Run the game

Save the replacement and run:

```
cd /d C:\Users\dabbu\Desktop\12_maze_runner
python main.py
```

You should now see the **difficulty selection screen first**, rather than the maze opening immediately.

Feature	How to test
Difficulty	Click Easy, Medium, or Hard
Fog of War	Move around; only a circle around the player stays visible
BFS Hint	Press `H` to show/hide the orange shortest-path squares
Leaderboard	Reach the exit; your completion time appears in the top 5
Persistence	Restart the application; previous leaderboard scores remain
Reset	Press `R`
Difficulty menu	Press `Esc`

Important: I inspected your original `maze.py` and `player.py`. This replacement is compatible with their existing function signatures, but `player.py` has a pre-existing limitation: it does not properly enforce internal maze-wall collisions. That is separate from these four features and may affect how the game plays.

For the four requested tasks, **replace only** `game/game_engine.py`.

You said:

can we fix the pre existing limitation so the player cannot cross the walls

Worked for 1m 1s

ChatGPT said:

Yes! I found the exact problem in your original `player.py`.

Your existing `_hits_wall()` function only checks whether the player's corners are outside the maze. **It never checks whether an internal wall separates two cells.** That's why the player can move straight through walls.

We can fix this by checking collisions against the maze's actual north, south, east, and west wall segments.

File to replace

`12_maze_runner/game/player.py`

No changes are required to `game_engine.py`, `maze.py`, or `main.py`. All four previously implemented features can remain as they are.

Complete replacement code

```
import pygame
from game.maze import CELL

SPEED = 3


class Player:
    def __init__(self, r, c):
        self.r = r
        self.c = c

        x = c * CELL + CELL // 2
        y = r * CELL + CELL // 2

        self.rect = pygame.Rect(
            x - 10,
            y - 10,
```

```
            20,
            20
        )

        self.color = (60, 120, 220)

    def move(self, keys, walls, rows, cols):
        dx, dy = 0, 0

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            dx = -SPEED

        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            dx = SPEED

        if keys[pygame.K_UP] or keys[pygame.K_w]:
            dy = -SPEED

        if keys[pygame.K_DOWN] or keys[pygame.K_s]:
            dy = SPEED

        # Move horizontally first
        if dx != 0:
            new_rect = self.rect.move(dx, 0)

            if not self._hits_wall(
                new_rect, walls, rows, cols
            ):
                self.rect = new_rect

        # Move vertically second
        if dy != 0:
            new_rect = self.rect.move(0, dy)

            if not self._hits_wall(
                new_rect, walls, rows, cols
            ):
                self.rect = new_rect

        # Update current cell coordinates
        center_x, center_y = self.rect.center

        self.c = center_x // CELL
        self.r = center_y // CELL
```

```
    def _hits_wall(self, rect, walls, rows, cols):
        """
        Check whether the player's rectangle
        crosses any maze wall.

        Wall indices:
        0 = North
        1 = South
        2 = East
        3 = West
        """

        # Check outer maze boundaries
        if (
            rect.left < 0
            or rect.top < 0
            or rect.right > cols * CELL
            or rect.bottom > rows * CELL
        ):
            return True

        # Cells overlapped by player's rectangle
        left_col = rect.left // CELL
        right_col = (rect.right - 1) // CELL

        top_row = rect.top // CELL
        bottom_row = (rect.bottom - 1) // CELL

        # Check vertical wall segments
        # between horizontally adjacent cells
        for c in range(left_col + 1, right_col + 1):
            boundary_x = c * CELL

            if not (rect.left < boundary_x < rect.right):
                continue

            for r in range(top_row, bottom_row + 1):

                # East wall of left cell
                east_wall = walls[r][c - 1][2]

                # West wall of right cell
                west_wall = walls[r][c][3]

                if east_wall or west_wall:
```

```
                return True

        # Check horizontal wall segments
        # between vertically adjacent cells
        for r in range(top_row + 1, bottom_row + 1):
            boundary_y = r * CELL

            if not (rect.top < boundary_y < rect.bottom):
                continue

            for c in range(left_col, right_col + 1):

                # South wall of upper cell
                south_wall = walls[r - 1][c][1]

                # North wall of lower cell
                north_wall = walls[r][c][0]

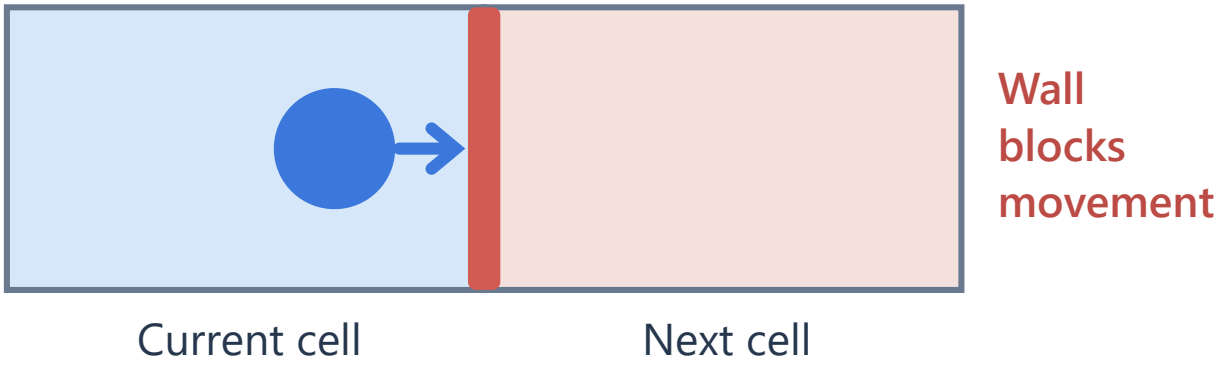
                if south_wall or north_wall:
                    return True

        return False

    def draw(self, screen):
        pygame.draw.ellipse(
            screen,
            self.color,
            self.rect
        )
```

How the collision detection works

Suppose the player attempts to move right:



The player's rectangle cannot overlap a closed wall segment.

If the east wall is closed, horizontal movement is rejected. If that passage is open, the player can move into the next cell.

Horizontal and vertical movement are checked separately, so diagonal movement won't let the player cut through a wall.

Final steps

The replacement file passed Python syntax validation. I haven't tested movement interactively in Pygame.

You can copy the code above or download the updated player.py ↗.

Save it at:

```
C:\Users\dabbu\Desktop\12_maze_runner\game\player.py
```

Then run:

```
python main.py
```

Try walking into a wall, through an open passage, and diagonally near a corner.

Expected result: The player stops at closed walls, moves through open passages, and cannot escape the maze boundaries.

player.py
Code